

Question B: Modeling with UML State Machines (60 marks)

Specify a **UML state machine** for a (simplified) digital chess clock based on the problem description below. Show the states as well as transitions with events, guards and actions. User interface events are predefined, while guards and actions may require parameters. Define at least two regions for the state machine. If you use private methods for guards or actions in your state machine, use **meaningful** names for those methods. You do **not** need to define the method bodies of those methods, **you only need to define their interface** (i.e. name, parameters, and return value, if any).

Digital Chess Clock (DCC)



The digital chess clock has six buttons (each of which corresponds to one event): **flip** (the large white button on the top), **minus**, **plus**, **start/stop**, **select** (the four blue buttons from left to right), **on/off** (at the bottom, not shown in figure).

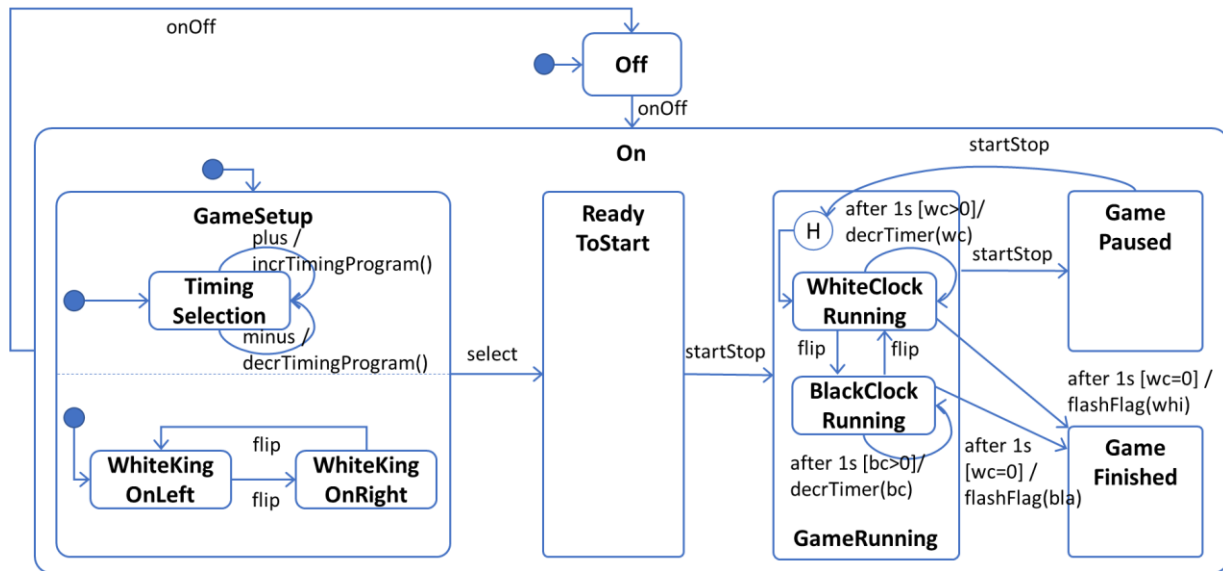
After turning on the chess clock (with the **on/off** button), players can iterate through all the (predefined) timings using the **plus** and **minus** buttons, and finally select the designated timing by the **select** button. A timing option has a predefined base time (minutes and seconds) and an increment (in seconds, which can be zero). The game then starts when the **start/stop** button is pressed.

At any time before the game is started, the players can set the clock to match the actual seating of White and Black players using the **flip** button (e.g. see the placement of the small symbols with the White and Black kings on the screen which can be swapped by pressing the **flip** button).

When the game is started, White's clock starts counting down (in each second) until the **flip** button is pressed. At this moment, White's clock stops (and receives a bonus time defined by the increment in the timing option), and Black's clock starts counting down. If the **flip** button is pressed again, then the same procedure is applied with reversed colors. Both clocks can be stopped by pressing the **start/stop** button, while the game can be continued by pressing the **start/stop** button again. If the clock of the current player counts down to zero, then a flashing flag shall appear on the screen.

The chess clock can be turned off at any time by pressing the **on/off** button.

a) Draw your UML state machine here



WhiteClockRunning.entry: startTimer(wc)
 WhiteClockRunning.exit: stopTimer(wc)
 BlackClockRunning.entry: startTimer(bc)
 BlackClockRunning.exit: stopTimer(bc)